CÓDIGOCLARO · MATERIAL DE ESTUDIO

Simulacros DAM: integración de módulos

Casos de examen con XML, XPath, SQL, PL/SQL, Java y desarrollo web

1. Preparar un simulacro integrado

Objetivo: Organizar un ejercicio de varias asignaturas antes de comenzar a programar y entregar evidencias verificables.

Un caso integrado suele describir una entidad, datos de entrada, almacenamiento y una interfaz. Primero identifica requisitos y entrega cada resultado en el formato solicitado: esquema, consulta, rutina, programa o formulario. Dibuja las relaciones y define claves antes de escribir SQL. Anota los datos límite que podrían romper una solución: lista vacía, identificador inexistente, XML sin namespace esperado, número inválido o respuesta HTTP fallida. Divide el tiempo en lectura, implementación, ejecución y revisión. Al terminar, presenta evidencia reproducible: datos de prueba, entrada, salida esperada y explicación breve de la decisión más importante.

Código · ejemplo

```
[Plan de trabajo]
1. Extraer entidades y reglas
2. Diseñar claves y formatos
3. Implementar por módulos
4. Probar casos normales y frontera
5. Revisar la salida y documentar
```

Asigna tiempo a lectura, diseño, implementación, comprobación y entrega. Antes de programar, escribe una fila de ejemplo y un caso límite; ambos sirven para comprobar que XML, tabla, objeto Java y formulario usan la misma identidad. Separa lo que pide el enunciado de la tecnología elegida. La evidencia final debe permitir a otra persona repetir la consulta o ejecutar el ejemplo sin datos personales ni dependencias no declaradas.

Práctica: El caso da cursos, alumnado y matrículas. ¿Qué resultado planeas antes de abrir el editor?

> Respuesta razonada: Lista entidades, relaciones y operaciones pedidas; anota una fila válida y un caso sin matrícula. Decide qué datos están en XML, SQL y objeto Java, y cómo se identifica cada entidad. Ese pequeño contrato orienta el modelo y previene que cada módulo use nombres o tipos distintos.

Documentación consultada: Oracle AI Database 26ai · SQL Language Reference

2. XML de matrícula y consulta XPath

Objetivo: Validar una entrada XML de matrículas y seleccionar los registros cuyo estado y nota cumplan el enunciado.

Supón un documento XML con namespace `urn:dam:matriculas` y matrículas con estado y nota. La expresión siguiente es XPath 3.1, no un prólogo XQuery: la aplicación enlaza el prefijo `m` a la URI antes de evaluar. Comprueba que el documento esté bien formado, que nota sea decimal y que la condición se aplique solo a matrículas evaluadas. La ruta devuelve nodos que coinciden; si no hay resultados, la secuencia queda vacía. El prefijo puede ser distinto al escrito físicamente en el XML siempre que apunte a la misma URI.

Código · XPath 3.1

```
//m:matricula[@estado = "evaluada"][xs:decimal(m:nota) ge 5]
```

Prueba también `count(//m:matricula)` con cero, una y varias matrículas. Si el documento emplea un namespace, configura el prefijo en la API; una ruta sin binding puede parecer correcta y devolver vacío. Conserva XML mínimo y resultado esperado como evidencia del simulacro.

Práctica: ¿Qué ocurre si nota contiene `apta` y por qué esta expresión debe evaluarse con el namespace configurado?

Respuesta razonada: La conversión directa a decimal falla si el contenido no es numérico. Valida el dato con XSD o comprueba `castable as xs:decimal` y define cómo tratar el registro inválido. Sin enlazar m a `urn:dam:matriculas`, XPath buscaría otro nombre expandido y podría devolver una secuencia vacía. Un estado pendiente tampoco tiene por qué contener nota.

Documentación consultada: W3C · XML 1.0 · W3C · XPath 3.1

3. Modelo relacional y consulta de aprobados

Objetivo: Traducir alumnado, módulos y matrículas a claves relacionales y consultar datos sin duplicados accidentales.

Un diseño normalizado separa alumno, módulo y matrícula; la matrícula actúa como entidad asociativa cuando un alumno cursa varios módulos y cada módulo tiene muchos alumnos. Claves primarias identifican filas y claves foráneas preservan la relación. Declara si existe una sola matrícula por pareja alumno-módulo con una restricción única. Una consulta con JOIN debe especificar las columnas de unión, no confiar en coincidencias de nombres. `DISTINCT` puede ocultar una unión incorrecta, por lo que primero confirma cardinalidades. Usa aliases y selecciona solo campos necesarios. El SQL de ejemplo emplea sintaxis estándar cercana a Oracle; adapta tipos a la versión del motor.

Código · ejemplo

```
[SQL · Oracle AI Database 26ai]
SELECT a.nombre, m.nombre AS modulo
FROM alumno a
JOIN matricula x ON x.alumno_id = a.id
JOIN modulo m ON m.id = x.modulo_id
WHERE x.nota >= 5;
```

Una clave primaria identifica la fila de alumno; una clave foránea impide que matrícula apunte a un alumno inexistente. La consulta de aprobados agrupa por la unidad que define el informe y filtra con HAVING cuando el criterio depende de AVG o COUNT. Comprueba que un alumno con varias matrículas no multiplique la media de forma inesperada. Ejecuta un caso con una sola nota y otro módulo sin resultados.

Práctica: ¿Cómo detectarías si el resultado duplica un alumno por tener varias matrículas aprobadas?

> Respuesta razonada: Define la granularidad del resultado: aquí una fila representa una matrícula aprobada, no necesariamente una persona única. Si piden alumnos, usa `EXISTS` o una agrupación adecuada; no añadas `DISTINCT` sin decidir si se debe mostrar cada módulo. Prueba un alumno con dos módulos aprobados para observar la cardinalidad.

Documentación consultada: Oracle AI Database 26ai · SQL Language Reference

4. SQL: cambios atómicos de matrícula

Objetivo: Proteger una operación que registra una matrícula y descuenta una plaza disponible dentro de una transacción.

La operación modifica al menos la matrícula y el número de plazas. Si una parte se confirma y la otra falla, el estado queda incoherente. Agrupa los cambios en una transacción y define qué tabla es fuente de verdad para disponibilidad. Un patrón condicional actualiza solo si quedan plazas; revisa el número de filas afectadas antes de insertar. La restricción única evita matrículas duplicadas aun bajo concurrencia. Parámetros enlazados previenen errores de comillas y reducen riesgos de inyección. SQL de Oracle distingue secuencias y sintaxis de otros motores, así que usa el dialecto indicado en el examen.

Código · ejemplo

```
[SQL · Oracle AI Database 26ai]
UPDATE modulo
SET plazas = plazas - 1
WHERE id = :modulo_id AND plazas > 0;
-- comprobar SQL%ROWCOUNT antes de insertar
```

Una reserva de plazas cambia una cantidad y crea una matrícula; si solo una operación confirma, el sistema deja datos incoherentes. Agrupa ambas en la transacción, valida filas afectadas y usa ROLLBACK si falla una regla. Usa placeholders del controlador para valores y deja que el driver los enlace; no concatenes cadenas recibidas del formulario. Prueba una inscripción duplicada y una plaza agotada en una base de pruebas.

Práctica: ¿Cuándo insertas la matrícula y cómo respondes si se actualizaron cero filas?

> Respuesta razonada: Inserta solo si la actualización afectó exactamente una fila. Si afectó cero, no quedaban plazas o el módulo no existe; no insertes la matrícula y devuelve el resultado de negocio correspondiente. La transacción y restricciones únicas protegen consistencia adicional; el bloqueo y nivel de aislamiento deben probarse con concurrencia real.

Documentación consultada: Oracle AI Database 26ai · SQL Language Reference

5. PL/SQL: cursor para emitir certificados

Objetivo: Recorrer matrículas aprobadas y producir una salida sin perder errores ni dejar la transacción implícita.

Una rutina que genera certificados puede consultar aprobados por curso y recorrer filas. Usa un cursor FOR LOOP si el recorrido es necesario y procesa una fila cada vez; no incluyas `COMMIT` por iteración. Si la salida solo es un conjunto de datos, una consulta SQL puede ser mejor que PL/SQL. Define qué ocurre con alumnos sin correo o certificados ya emitidos y protege la operación con una clave única o idempotencia. El cliente de examen debe indicar si espera `DBMS_OUTPUT`, filas devueltas o una tabla de emisión. Comprueba que el entorno habilite la salida antes de depender de ella.

Código · ejemplo

```
[PL/SQL · Oracle AI Database 26ai]
BEGIN
  FOR r IN (SELECT alumno_id FROM matricula WHERE nota >= 5) LOOP
    DBMS_OUTPUT.PUT_LINE('Aprobado: ' || r.alumno_id);
  END LOOP;
END;
/
```

Un cursor explícito sirve para procesar múltiples filas en PL/SQL, pero una consulta de conjunto es más simple cuando basta con insertar o actualizar todos los registros. Comprueba `%NOTFOUND` tras FETCH antes de usar variables y cierra el cursor. Si el informe no tiene alumnos, el bloque debe comunicar “sin resultados” o producir cero certificados según el contrato, no emitir una fila anterior que quedó en memoria.

Práctica: ¿Por qué este bloque no demuestra todavía que un certificado se haya guardado?

> Respuesta razonada: `DBMS_OUTPUT` solo escribe una línea en el buffer de salida de la sesión; no inserta ni confirma un certificado. Para persistir, hace falta una tabla y una operación DML, con claves idempotentes y una política de transacción. La salida de consola sirve para revisión, no como prueba de persistencia.

Documentación consultada: Oracle AI Database 26ai · PL/SQL Language Reference

6. Java: agrupar notas con colecciones

Objetivo: Calcular una media por alumno y tratar explícitamente faltantes, precisión y claves de colección.

Los mapas facilitan asociar a cada alumno una colección de notas; una solución clara puede usar `Map<Long, List<BigDecimal>>`. Define si las notas nulas se descartan o se rechazan y si deben estar en un rango. Con dinero o notas decimales, `BigDecimal` evita aproximaciones binarias, pero la escala y modo de redondeo siguen siendo decisiones del dominio. No asumas que el orden de un `HashMap` es estable. Un DTO inmutable ayuda a entregar resultado coherente. La prueba debe cubrir alumno sin notas, una nota y varias notas con media repetible.

Código · ejemplo

```
[Java 21]
BigDecimal average(List<BigDecimal> values) {
    return values.stream().reduce(BigDecimal.ZERO, BigDecimal::add)
        .divide(BigDecimal.valueOf(values.size()), 2, RoundingMode.HALF_UP);
}
```

Map agrupa notas por clave de módulo; getOrDefault inicializa el contador o suma en cero sin una rama separada. Define qué ocurre si la nota está ausente y valida que no se añada un null a la media. ArrayList conserva posiciones; HashMap no ordena claves para el informe. Si se necesita salida determinista, ordena por código antes de presentar. Calcula a mano dos notas de un módulo para validar el acumulador.

Práctica: ¿Qué caso falla en este método si `values` está vacío?

Respuesta razonada: La división usa divisor cero y lanza `ArithmeticException`. Decide si no hay media debe producir `Optional.empty()`, un error de dominio o una convención documentada; no inventes cero, pues cero sería una media real. Valida también nulos y fija escala y redondeo.

Documentación consultada: Oracle · Java SE 21 Collections API · Oracle · Java Language Specification 21

7. HTML: formulario accesible de matrícula

Objetivo: Crear un formulario semántico que asocie etiquetas, valide datos básicos y comunique errores con claridad.

Un formulario requiere etiquetas vinculadas a controles, nombres descriptivos y botones cuyo propósito esté claro. Los atributos HTML como `required`, `type="email"` o `min` ayudan al usuario, pero no sustituyen validación en servidor. Para accesibilidad, asocia `label` por `for`/`id`, comunica instrucciones y errores sin depender solo del color, y conserva foco utilizable con teclado. El botón de envío debe tener tipo explícito cuando hay otros botones en el formulario. No pidas datos que la operación no necesite. Prueba con lector de pantalla o navegación por teclado y con datos inválidos.

Código · ejemplo

```
[HTML]
<label for="email">Correo del alumno</label>
<input id="email" name="email" type="email" required>
<button type="submit">Matricularme</button>
```

La etiqueta debe nombrar el control y el atributo name identifica el campo enviado. required aporta una comprobación cliente, no autorización ni validación final. Muestra el error junto al campo y conserva los valores corregibles. Tab debe recorrer controles en un orden lógico y el foco debe ser visible. Si el formulario de matrícula se envía a un servidor, vuelve a comprobar identificador, plazas y permisos antes de persistir.

Práctica: ¿Por qué el servidor debe volver a validar un correo marcado como obligatorio en HTML?

> Respuesta razonada: La persona puede desactivar o saltarse las restricciones del navegador y enviar una petición modificada. El servidor es la autoridad que valida formato, tamaño, autorización y reglas de negocio. La validación del cliente mejora la experiencia y da errores tempranos; no es una frontera de seguridad.

Documentación consultada: WHATWG · HTML Living Standard · W3C WAI · Forms

8. JavaScript: petición y estados de interfaz

Objetivo: Solicitar datos de módulos con `fetch`, verificar estado HTTP y representar carga, error y éxito.

Una interfaz que consulta plazas debe distinguir tres estados visibles: cargando, resultado y error. `fetch` devuelve una promesa; debe esperarse la respuesta y comprobar `response.ok`, porque 404 o 500 no siempre rechazan esa promesa. Después se analiza JSON y se valida su forma antes de mostrar campos. Un `try/catch` contextualiza fallos de red, estado HTTP y análisis. Evita insertar texto remoto con `innerHTML`; usa `textContent` si solo quieres mostrar texto. Si pueden solaparse solicitudes, considera cancelación con `AbortController` para evitar que una respuesta antigua reemplace una nueva.

Código · ejemplo

```
[JavaScript · ECMAScript 2025]
async function cargarModulos() {
  list.textContent = 'Cargando…';
  try {
    const response = await fetch('/api/modulos');
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    const modulos = await response.json();
    if (!Array.isArray(modulos) ||
        !modulos.every(m => m && typeof m.nombre === 'string')) {
      throw new Error('Formato inválido');
    }
    list.textContent = modulos.length
      ? modulos.map(m => m.nombre).join(', ')
      : 'No hay módulos.';
  } catch {
    list.textContent = 'No se pudieron cargar los módulos.';
  }
}
void cargarModulos();
```

Comprueba response.ok, analiza JSON y valida los campos antes del estado listo: un 200 no certifica el formato. Representa carga, error y datos en estados distintos para no mostrar información obsoleta. Cancela o ignora peticiones anteriores cuando cambie el filtro, y prueba fallo de red, 404 y campos ausentes.

Práctica: ¿Qué riesgo hay en asumir que `modulos` es siempre un array válido?

> Respuesta razonada: La respuesta puede tener otro formato, estar vacía o provenir de un servidor con error de contrato. Comprueba `Array.isArray` y valida los campos requeridos antes de usarlos. Mantén salida segura con `textContent`, muestra un mensaje útil en el estado de error y no expongas detalles internos del servidor.

Documentación consultada: ECMA · ECMAScript 2025 · WHATWG · Fetch Standard

9. Depurar una integración entre módulos

Objetivo: Seguir un dato desde la entrada hasta el resultado y localizar el primer punto donde cambia incorrectamente.

Cuando un certificado muestra una nota equivocada, traza el mismo identificador en XML, SQL, PL/SQL, Java y la interfaz. Comprueba en cada frontera tanto representación como tipo: `8.5` como decimal, cadena JSON o `BigDecimal` no son intercambiables sin conversión. Añade un caso mínimo de reproducción y registra solo datos permitidos. Verifica que el JOIN no multiplique filas, que el cursor reciba las condiciones correctas y que la API devuelva el HTTP esperado. Aísla fallos por capas antes de modificar varias piezas a la vez. Después de corregir, añade una prueba de regresión que falle con el defecto anterior.

Código · ejemplo

```
[Lista de comprobación]
Entrada → validación → consulta → resultado → serialización → presentación
```

La traza se reduce al primer límite donde difiere el valor esperado: compara identificador, tipo y cardinalidad en cada paso. No cambies varias capas simultáneamente; una reproducción mínima permite saber si el fallo está en el JOIN, la conversión o el renderizado.

Práctica: La base de datos tiene la nota correcta, pero la web muestra otra. ¿Qué fronteras revisarías primero?

> Respuesta razonada: Compara el registro por su ID en la consulta y su proyección, revisa la conversión a DTO y el JSON serializado, y observa qué respuesta recibió el navegador. Si el servidor devuelve el dato correcto, inspecciona estado y renderizado del cliente; si no, retrocede al JOIN y a la condición SQL. Cambia una sola frontera y reproduce.

Documentación consultada: Oracle AI Database 26ai · SQL Language Reference · Oracle AI Database 26ai · PL/SQL Language Reference

10. Entrega del simulacro y autoevaluación

Objetivo: Presentar código reproducible y revisar corrección, robustez, claridad y adecuación al dialecto solicitado.

Entrega los archivos solicitados y una explicación breve de decisiones. Anota motor y versión para SQL/PLSQL, Java y runtime si afectan a la compatibilidad. Incluye datos mínimos reproducibles, salida esperada y resultado observado. Evalúa contrato, modelo, implementación, pruebas y explicación con 2 puntos por respuesta completa, 1 por parcial y 0 por incorrecta: con 8/10 o más pasa a práctica; con menos de 6/10 repasa y repite justificando los errores. Elimina credenciales y datos reales del alumnado antes de entregar.

Rúbrica: 2 puntos para contrato y límites; 2 para XML/XPath; 2 para SQL/PLSQL; 2 para Java/Web; 2 para pruebas y explicación.

Código · java

```java
int score = 7;
String plan = score >= 8 ? "apto" :
    score >= 6 ? "revisar" : "repetir";
```

El ejercicio puede incluir cinco respuestas cortas: XPath, SELECT INTO, Comparable, LEFT JOIN y await. Puntúa cada una, explica el error con una fuente y repite solo las que no alcanzan el contrato. Guarda la hoja de resultados para comparar el segundo intento.

Práctica: Puntúa 7/10 con una respuesta incompleta sobre LEFT JOIN. ¿Qué debe revisar la persona antes de repetir el simulacro?

> Respuesta razonada: 7/10 queda entre los umbrales: revisa la respuesta parcial, consulta la documentación del motor y practica un caso con alumno sin matrículas. La evidencia distingue una consulta correcta de una que solo compila: datos con cardinalidad conocida, resultado esperado y observado, incluyendo cero coincidencias y duplicados. Explica por qué el JOIN conserva o descarta la fila.

Documentación consultada: W3C · XPath 3.1 · Oracle AI Database 26ai · PL/SQL Language Reference · Oracle · Java Language Specification 21 · WHATWG · HTML Living Standard

Fuentes oficiales

W3C · XML 1.0 · W3C · XPath 3.1 · Oracle AI Database 26ai · SQL Language Reference · Oracle AI Database 26ai · PL/SQL Language Reference · Oracle · Java Language Specification 21 · WHATWG · HTML Living Standard · ECMA · ECMAScript 2025 · W3C WAI · Forms · W3C · XQuery 3.1 · Oracle · Java SE 21 Collections API · WHATWG · Fetch Standard